

OPTIMIZED CUDA IMPLEMENTATION OF THE BINOMIAL OPTION PRICING MODEL

Ruben Ciranni, Nicolas Dickenmann, Luigi Pizza, Giovanni Spadaccini, Alessandro Tazza

Department of Computer Science
ETH Zürich
Zürich, Switzerland

ABSTRACT

The Binomial Option Pricing Model, a numerical method for pricing American options, suffers from quadratic computational complexity, limiting its use in real-time applications. While Graphics Processing Units (GPUs) offer significant theoretical throughput, existing implementations face synchronization bottlenecks, memory bandwidth limitations, or rigid hardware constraints that restrict the number of discretization steps. This paper presents an optimized CUDA implementation¹ of the BOPM that delivers state-of-the-art performance by maximizing hardware utilization across all operational regimes: for single-option pricing at $N = 16,384$ time steps, our solver achieves a $736\times$ speedup over the industry-standard QuantLib CPU baseline and a $133\times$ speedup over naive GPU implementations. In bulk pricing scenarios, the method reaches a peak throughput of 185 KOptions/s, outperforming NVIDIA’s optimized reference implementation by a factor of 15.

1. INTRODUCTION

Motivation. The distinctive ability to exercise American options prior to maturity precludes closed-form pricing solutions like Black-Scholes-Merton [1], necessitating the use of numerical methods. The standard Cox-Ross-Rubinstein [2] Binomial Option Pricing Model (BOPM) is one such example, but scales quadratically with the number of discretization steps, making high-precision CPU valuation prohibitively slow. While GPUs naturally provide the throughput for large-scale portfolio analysis, achieving the low latency required for real-time pricing is challenging due to recursive lattice dependencies that bottleneck naive implementations due to synchronization and memory bandwidth constraints. We demonstrate that an optimized GPU solver can simultaneously deliver the massive throughput for offline analysis and the ultra-low latency essential for live trading.

Related Work. Among CPU implementations of the BOPM, the QuantLib [3] library is a standard industry baseline for correctness and performance. Early GPU attempts

by Kolb and Pharr [4] parallelized individual lattice layers via separate kernel launches, mapping threads to lattice nodes. However, this approach incurs substantial overhead due to repeated kernel invocations and the excessive global memory traffic required to persist state between steps. Consequently, their reported $10\times$ speedup relative to a naive CPU baseline on a lattice with 1,024 time steps is derived entirely from the aggregate throughput of computing multiple independent trees in parallel, rather than accelerating individual valuations. Their method was evaluated on a NVIDIA GEFORCE 6800, AMD Athlon 64 3200+. In the official NVIDIA CUDA Samples implementation, Podlozhnyuk [5] proposed an optimized framework in which each block processes several backward steps of the binomial tree at once by repeatedly reducing a window of values using a double-buffered shared-memory scheme. While effective, this method is bounded by hardware constraints: the number of discretization steps cannot exceed the maximum number of threads per block (1,024 on most architectures at present), and the volume of parallel options is limited by available shared memory capacity. Podlozhnyuk reported no setup details or performance metrics. Ganesan et al. [6] focused on accelerating the computation of a single option price as opposed to bulk option pricing. Their approach partitions the binomial lattice along the temporal axis, overcoming the inherent sequential dependency of the dynamic programming recurrence by introducing symbolic coefficients that relate non-adjacent time steps. Although source code is not available, the authors report an observed speedup of $2\times$ over a naive GPU baseline, when evaluated on an NVIDIA GeForce 8600GT GPU using a tree with 1,024 time steps.

Our work advances these contributions with novel optimizations that approach the theoretical hardware limits of modern GPU architectures. Unlike prior implementations, our strategy delivers exceptional results for both high-throughput bulk option pricing and low-latency single-option valuation.

2. BACKGROUND

The American Option. An option [7] is a financial con-

¹Source code is available at <https://github.com/rubenciranni/option-pricing-cuda>

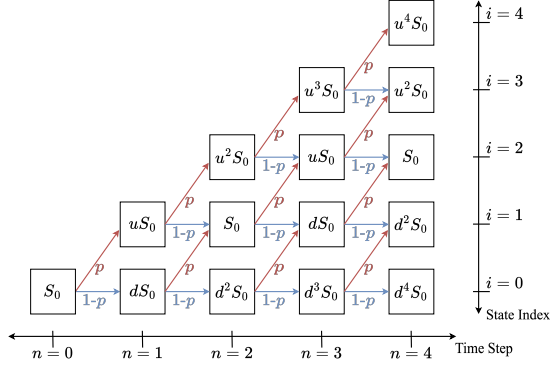


Fig. 1. Lattice structure of the BOPM with $N = 4$ time steps. Each node represents a possible asset price at a given time, and any path from the root to a terminal node traces one possible evolution of the underlying asset price over the contract lifetime.

tract granting the holder the right to buy (*call* option) or sell (*put* option) an underlying asset at a fixed price K (the *strike*) on or before an expiration date T . Unlike *European options*, which are exercisable only at T , *American options* allow *early exercise* at any time. This added flexibility precludes simple closed-form pricing solutions like Black-Scholes [1], instead requiring numerical methods.

The Binomial Option Pricing Model. The Cox-Ross-Rubinstein [2] Binomial Option Pricing Model (BOPM) approximates continuous asset dynamics by discretizing the option's lifespan T into N intervals of length $\Delta t = T/N$. Prices evolve on a price lattice where movements occur with risk-neutral probabilities p (upward) and $1 - p$ (downward), as depicted in Figure 1. Computationally, the lattice is strictly recombining, meaning an upward movement followed by a downward movement yields the same value as the reverse. This property restricts the total unique nodes to $\frac{1}{2}N(N+1)$, rendering the method tractable.

The model relies on the input tuple $(S_0, K, T, \sigma, r, q, N, s)$. S_0 and K denote the initial asset price and strike price, respectively, while T represents the time to maturity. The risk-free rate r and dividend yield q govern discounting and drift, while volatility σ determines the diffusion intensity. The discretization granularity is defined by N , with convergence to the continuous-time value as $N \rightarrow \infty$. Finally, the option sign $s \in \{\pm 1\}$ serves as a binary indicator for put ($s = -1$) and call ($s = +1$) options.

These inputs define the lattice dynamics visualized in Figure 1. The movement factors are calculated as $u = \exp(\sigma\sqrt{\Delta t})$ and $d = u^{-1}$, ensuring logarithmic symmetry. The corresponding risk-neutral probability for an upward movement is given by $p = (e^{(r-q)\Delta t} - d)/(u - d)$. This parameterization ensures asymptotic consistency [2]: as $N \rightarrow \infty$ (and consequently $\Delta t \rightarrow 0$), the discrete bino-

mial process converges weakly to the continuous Geometric Brownian Motion defined by the Black-Scholes [1] stochastic differential equation.

Valuation Algorithm. The valuation proceeds via initialization followed by backward induction. We index nodes by time n and state i , with the *underlying* asset price given by the closed-form $S_{n,i} = S_0 u^{2i-n}$ (see Figure 1). First, terminal payoffs at maturity N are given by:

$$V_{N,i} = \max \{s \cdot (S_{N,i} - K), 0\} \quad (1)$$

as the option can either be exercised yielding a profit of $s \cdot (S_{N,i} - K)$ or expire worthless. Subsequently, the algorithm iterates backward from $n = N - 1$ to 0. At each node, the option value is determined by maximizing between the intrinsic value of the option (exercise value) and the expected continuation value (holding value):

$$V_{n,i} = \max \left\{ s \cdot (S_{n,i} - K), e^{-r\Delta t} [pV_{n+1,i+1} + (1-p)V_{n+1,i}] \right\} \quad (2)$$

Finally, the root node $V_{0,0}$ yields the fair market price.

3. PROPOSED METHOD

We now present our GPU-accelerated implementation of the Cox-Ross-Rubinstein [2] BOPM for American options. Through careful use of algorithmic optimizations, memory hierarchy exploitation, and numerical precision methods, we achieve substantial performance improvements over baseline implementations. Unlike previous attempts, our method delivers state-of-the-art performance for both high-throughput bulk pricing and low-latency single-option valuation.

Banked Exercise Value Caching. The exercise value $V^{\text{exercise}} = s \cdot (S_{n,i} - K) = s \cdot (S_0 \cdot u^{2i-n} - K)$ exhibits a structured dependence on the time-step n and state-index i that can be exploited. Since the exponent $2i - n$ ranges over exactly $2N + 1$ distinct integer values across the tree, all possible exercise values can be precomputed during an initialization phase and stored in a global lookup array.

The tree structure exhibits an important access pattern that can be exploited for memory optimization. As shown in Figure 1, nodes at odd-indexed time steps require exercise values corresponding to odd exponents, while even-indexed layers require even exponents. We therefore organize the lookup array into two separate banks, storing odd-exponent values in one bank and even-exponent values in the other. This banking scheme ensures contiguous memory access patterns within each warp, as threads processing a single layer access elements from only one bank with unit stride.

Pruning of Out-of-the-Money (OTM) Nodes. The binomial tree presents a monotonic property that enables pruning of unnecessary computations. For call options, terminal nodes may have zero expected payoff when the strike

price exceeds the underlying asset price. In the financial literature [7], this condition is referred to as being *out-of-the-money*. It can be shown that an interior node will also be out-of-the-money if it is only connected to out-of-the-money nodes. We exploit this fact by identifying the highest (for call options) or lowest (for put options) zero-valued node in the terminal layer through binary search over the precomputed exercise values, then avoiding computation for all nodes in the dependency cone below this threshold.

Temporal Unrolling and Overlapped Thread Blocks.

The parallelization strategy proposed by Kolb and Pharr [4] launches one kernel per time step, incurring high overhead for deep trees. Inspired by Ganesan et al. [6], we address this bottleneck through temporal unrolling. By configuring thread blocks to collectively compute multiple tree layers within a single kernel invocation, we amortize launch overhead and optimize memory hierarchy usage by retaining intermediate results in registers and shared memory.

In this scheme, exemplified in Figure 2, each thread is assigned a temporally contiguous sequence of nodes at the same state-index i . Threads exchange node values using warp shuffle intrinsics and shared memory (for warp boundary values only), with warp-level or block-level barrier synchronization preventing race conditions. The shared memory usage per block is $\text{block_size}/\text{warp_size} \leq 1,024/32 \leq 32$ node values, which is $64\times$ less space than a double-buffer shared-memory-only approach like Ganesan et al. [6], requiring $2 \cdot \text{block_size}$ node values in shared-memory. This allows us to use deeper binomial trees or to launch a greater volume of option pricing tasks in parallel while incurring a lower average memory latency.

Distinct from Ganesan et al. [6], our computation pattern creates inter-block redundancy, where boundary nodes may be recomputed by multiple thread blocks (e.g., node $V_{8,4}$ in Figure 2). We carefully tune the unrolling factor to balance this redundancy against the benefits of reduced thread divergence and improved memory access. For our selected unrolling factors, the performance gains substantially outweigh the cost of redundant boundary computations.

Batch Processing. We implement a batch processing kernel that prices multiple independent options within a single launch. The batch implementation assigns different options to separate thread blocks, maintaining computational independence while sharing GPU resources efficiently. For small to moderate tree sizes where a single option’s computation underutilizes the available streaming multiprocessors, batching dramatically improves throughput by increasing occupancy and maintaining high GPU utilization.

Hyperparameter Optimization and Adaptive Autotuning. Our implementation exposes two primary hyperparameters that significantly impact performance: the number of threads per block and the temporal unrolling factor. We employ the CUDA Occupancy API’s `cudaOccupan`

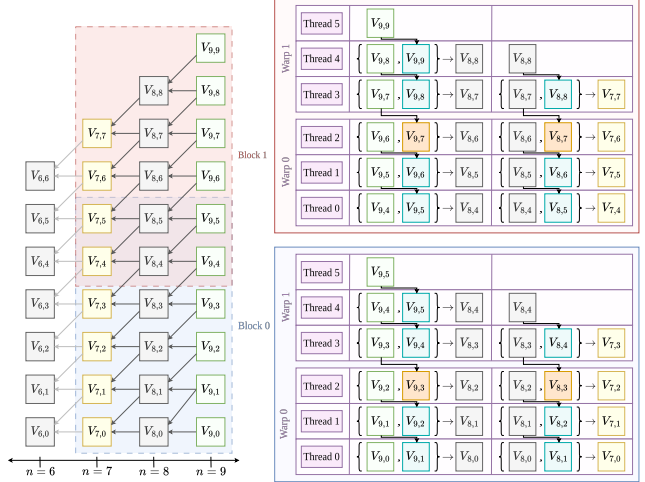


Fig. 2. Unrolled block computation with shuffle intrinsics for three consecutive layers of the binomial tree. Left: dependency structure with Block 0 (blue) processing $V_{9,0}$ through $V_{9,5}$ and Block 1 (red) processing $V_{9,4}$ through $V_{9,9}$. Right panels: sketched computation within each block containing two warps with six threads total. Each thread reads input values from global memory (green), computes intermediate values (white), and writes final results (yellow, e.g., $V_{7,0}$ through $V_{7,3}$ for Block 0) to global memory. Cyan boxes indicate shuffle-down communication within warps; orange boxes mark values requiring shared memory for cross-warp exchange. This organization has high occupancy and minimizes global memory traffic. Some nodes (e.g., $V_{8,4}$) are redundantly computed by multiple blocks.

`cyMaxPotentialBlockSize` function to explore the threads-per-block parameter space, identifying 256 threads per block as optimal for our kernel across all tested problem sizes. Conversely, we observe that the optimal temporal unrolling factor varies with problem size. Thus, we implemented an adaptive autotuning strategy that chooses the appropriate unrolling factor based on the remaining tree depth, launching different optimized kernels as the backward induction proceeds through layers of varying width.

Emulated-Double Arithmetic. Our target hardware platform, the NVIDIA RTX 5060 Ti, exhibits a 64:1 ratio between float (FP32) and double (FP64) performance [8]. Direct implementation using native FP32 arithmetic proves insufficient, as the 24-bit effective precision of single-precision floating-point cannot maintain acceptable accuracy for trees with depth exceeding several thousand levels, where rounding errors accumulate through repeated backward induction computations.

We address this precision-performance trade-off by implementing emulated-double arithmetic [9], in which each

logical value is represented as an unevaluated sum of two FP32 numbers. By implementing custom kernels for conversion, addition, and multiplication, we achieve 48 bits of effective precision, positioning our implementation between the 24-bit mantissa of standard FP32 and the 53-bit mantissa of native FP64. While this approach increases the total instruction count, the 64:1 hardware throughput advantage of FP32 on our target architecture provides ample headroom for significant net performance gains.

We evaluate numerical stability using a theoretical forward error analysis for recursive accumulation based on Higham [10]. For algorithms exhibiting linear error growth, the accumulated worst-case relative error at depth N is bounded by $E_{total} \leq C \cdot N \cdot \epsilon_{eff}$, where C represents the operations per logical step and ϵ_{eff} is the machine epsilon. In our specific implementation, each node executes exactly one multiplication, one fused multiply-add (FMA), and one max operation. Since the max operation does not introduce additional error, we set $C = 2$. Although the lattice size scales quadratically with depth, the error propagation follows a linear path. This yields a specific relative error bound of $E \approx 2 \cdot N \cdot \epsilon_{eff}$. Table 1 reports the accumulated error bound for $N = 100,000$, where our emulated double is accurate up to 8 decimal digits, compared to 10 decimal digits for native FP64.

Format	Eff. Bits	ϵ	Δ_{total}
FP32 (Single)	24	1.19×10^{-7}	2.38×10^{-2}
Emulated-Double	48	7.11×10^{-15}	1.42×10^{-9}
FP64 (Double)	53	2.22×10^{-16}	4.44×10^{-11}

Table 1. Relative accumulated error for different floating point formats for $N = 100,000$ (ϵ : machine epsilon; Δ_{total} : accumulated error bound).

4. EXPERIMENTAL RESULTS

Experimental Setup. Experiments are conducted on a single NVIDIA RTX 5060 Ti GPU and Intel Xeon E5-2680 V4 @ 2.4GHz CPU running Ubuntu 24.04.3 LTS. All code was compiled with the `-O3` optimization flag using `nvcc` (13.0) for device code and `g++` (13.3) for host code.

Evaluation Metrics. We evaluate performance of our optimized kernel across varying numbers of discretization steps N and batch sizes B using two primary metrics: execution time in milliseconds (ms), denoted as t , and computational throughput, denoted as P . As t naturally scales with the problem size, to quantify computational efficiency with varying B and N , we utilize the Giga-Nodes per second (GNodes/s) metric:

$$P_{node} = \frac{B \cdot N(N+1)}{2t} \cdot 10^{-6}.$$

By normalizing the total node count $\frac{1}{2}B \cdot N(N+1)$ against the runtime, this metric isolates the hardware’s effective processing rate from the size of the input. When comparing performance with a fixed N and varying B we instead measure the throughput P in Options per second (Options/s) as in Kolb and Pharr [4]:

$$P_{option} = \frac{B}{t} \cdot 10^3.$$

Our evaluation encompasses two distinct operational regimes: *single-option pricing* ($B = 1$), focusing on ultra-low latency, and *bulk option pricing* ($B \gg 1$), aiming for maximum aggregate throughput. To ensure statistical soundness, we report all measurements as $m [l; u]$ where m denotes the median of the sample followed by the corresponding 99% confidence interval (CI) $[l; u]$ in square brackets. These non-parametric CIs are computed using the rank-based method described by Le Boudec [11].

Baselines. We evaluate our optimized implementation against three distinct baselines. On the CPU side, we employ ① the widely used QuantLib [3] `VanillaBinomialEngine` as the industry standard for correctness and baseline performance. On the GPU side, we benchmark against ② a naive CUDA implementation based on Kolb and Pharr [4], which maps lattice nodes to threads with synchronization via global memory at each time step, and ③ the optimized solver provided in the NVIDIA CUDA Samples by Podlozhnyuk [5]. As noted in Section 1, baseline ③ is strictly constrained to $N \leq 1,024$, as N corresponds to the number of threads per block, which on our target architecture can be at most 1,024. For the same reason, baseline ③ performs best when N is a power of two.

Single-Option Pricing Execution Time. The execution time t of our method is compared against baselines ①, ②, and ③ in the single-option pricing task ($B = 1$). To ensure a rigorous comparison that favors the baselines (③ in particular), we restrict our evaluation to $N = 2^k$. Even under these conditions, our method demonstrates superior performance across the entire problem size range.

Figure 3 illustrates the execution time on a log-log scale. Our optimized kernel maintains the lowest execution time across all tested configurations. Notably, at $N = 1,024$, our method achieves an execution time of 0.28020 [0.27938; 0.28218] ms, outperforming the NVIDIA reference solver (③) by 23×, despite the latter being specialized for this specific range. As N increases to 16,384, the performance gap widens significantly: while the CPU baseline (①) degrades to nearly 2,345.1 [2,343.0; 2,347.2] ms and the naive GPU implementation (②) requires 425.21 [425.19; 425.24] ms, our solver completes the pricing in just 3.188 [3.186; 3.191] ms. This represents a speedup of approximately 133× over the naive GPU baseline and nearly 736× over the industry-standard CPU implementation. We recall that baseline ③

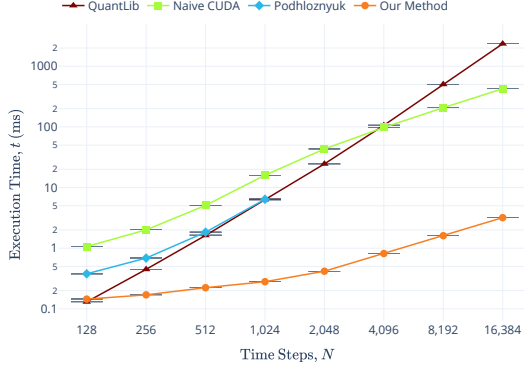


Fig. 3. Execution time (ms) as a function of the number of time steps on log-log scale, comparing our optimized implementation (orange) against baselines (Quantlib [3] in brown, naive CUDA based on Kolb and Pharr [4] in green, Podlozhnyuk [5] in blue) for the single-option pricing task ($B = 1$). Each data point represents the median of 50 independent runs following warm-up iterations. A 99% confidence interval is displayed for each data point, although always within 5% of the median. All measurements use identical option parameters: $S_0 = K = 100$, $T = 1$ year, $\sigma = 0.2$, $r = 0.03$, $q = 0.015$, with $s = -1$.

is constrained to $N \leq 1,024$, thus results are reported for that range only.

Bulk Option Pricing Throughput. The throughput P_{option} of our method is compared against the GPU baselines (2) and (3) in the bulk option pricing task ($B \gg 1$). As shown in Figure 4, our optimized implementation reaches a peak throughput of 185.21 [185.10, 185.41] KOptions/s. This corresponds to an improvement of $15\times$ over the CUDA Samples implementation (3) and $61\times$ over the naive CUDA baseline (2). While the baselines saturate earlier due to global memory bandwidth limitations, our method maintains high efficiency by leveraging the entire GPU memory hierarchy. We exclude QuantLib [3] from this experiment to ensure a fair comparison, as GPU batching provides an architectural advantage over serial CPU execution. However, we note that with $N = 1,024$ and $B = 1,024$, our method achieves $\approx 1,170\times$ the throughput of the CPU baseline.

Performance Scaling. We now examine how the performance of our method scales for varying batch sizes B and time steps N . As illustrated in Figure 5, our solver exhibits higher throughput as both N and B grow. For shallow lattices ($N \leq 1,024$), the kernel is latency-bound at low batch sizes; here, increasing B is critical to hiding global memory latency and instruction overhead, with throughput saturating only when $B \geq 4,096$. Conversely, for deep lattices ($N \geq 4,096$), the quadratic growth in arithmetic intensity ($O(N^2)$) ensures high occupancy even with modest

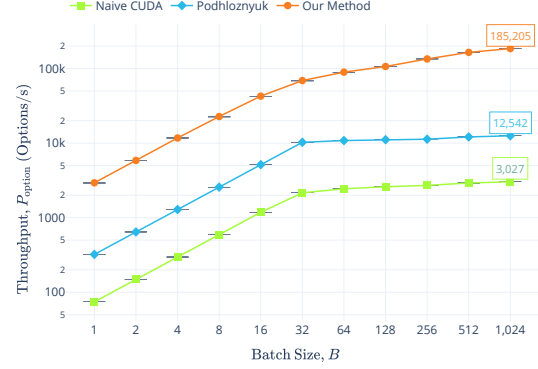


Fig. 4. Throughput (Options/s) as a function of the batch size on a log-log scale, comparing our optimized implementation (orange) against baseline methods (naive CUDA based on Kolb and Pharr [4] in green, Podlozhnyuk [5] in blue) for the bulk-option pricing benchmarks ($N = 1,024$). Each data point represents the median of 50 independent runs following warm-up iterations. A 99% confidence interval is displayed for each data-point, although always within 5% of the median. For a given batch size B , option parameters are sampled at random B times and kept constant for all measurements.

batch sizes. Notably, our adaptive autotuning strategy maintains consistent efficiency, preventing drastic performance drop-offs typically associated with shared memory oversubscription in naive baselines when N scales up.

Roofline Analysis. Figure 6 shows a roofline plot for our method in the bulk option pricing task. For $N = 16,384$ and $B = 1,000$ at an arithmetic intensity of approximately 18.644 [18.628; 18.653] FLOP/Byte, our implementation operates near the DRAM bandwidth ceiling, exhibiting memory-bound behavior with respect to main memory. The kernel achieves 76.541 [76.532; 76.555] % of peak compute throughput and 75.981 [75.974; 75.985] % of peak memory bandwidth according to NVIDIA Nsight Compute’s Speed of Light metrics [12]. For both L1 and L2 caches, the implementation is compute-bound, with L1 performance approaching the L1 roofline. This indicates effective data reuse within the cache hierarchy while main memory bandwidth is the primary performance limiter for our arithmetic intensity regime.

Ablation Study. To isolate the contribution of individual optimizations, we selectively disable each one of them and measure the resulting performance degradation relative to the fully optimized kernel. The ablation methodology is defined as follows: removing *Shuffle Intrinsics* forces a fallback to shared memory for inter-thread communication; disabling *Pruning of OTM Nodes* necessitates the evaluation of the complete lattice regardless of the out-of-the-money condition; without *Adaptive Autotuning*, thread-block con-

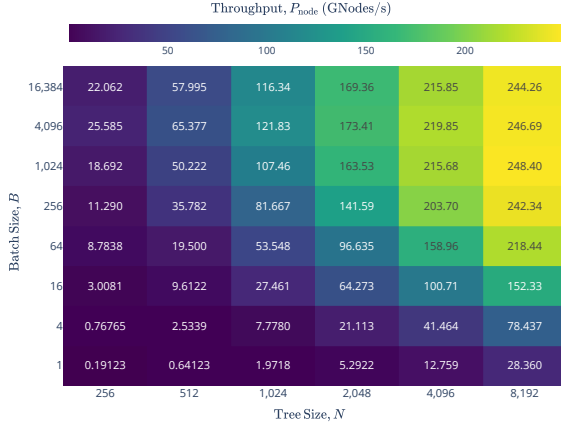


Fig. 5. Throughput (GNodes/s) of our method as a function of the number of time steps and the batch size. For a given batch size B , option parameters are sampled at random B times and kept constant for all measurements. Each reported value is computed from the median execution time of 50 independent runs following warm-up iterations. For the sake of readability, we don’t report a 99% confidence interval, as always within 5% of our reported median.

figurations and unroll factors are fixed to static defaults; disabling *Temporal Unrolling* sets the unroll factor to 1; replacing *Emulated-Double Arithmetic* involves reverting to native hardware double-precision; and removing *Exercise Value Caching* triggers the redundant recomputation of the intrinsic value at every node.

Table 2 presents the results. The most critical optimization is *Exercise Value Caching*, where removing the pre-computed lookup table results in a $120\times$ slowdown. *Emulated-Double Arithmetic* provides a substantial $3.8\times$ speedup over native double-precision, validating its use on hardware where FP64 units are scarce. Similarly, *Temporal Unrolling* contributes a $3.5\times$ improvement: by processing multiple lattice layers within a single kernel launch, this technique significantly amortizes the overhead of global memory usage and kernel invocation.

5. CONCLUSIONS

We presented an optimized CUDA implementation of the Binomial Option Pricing Model for American options, featuring optimizations such as exercise-value caching, emulated double-precision arithmetic, and temporal unrolling. Our implementation achieves a peak throughput of 185.21 KOptions/s in the bulk option pricing regime, representing a $15\times$ improvement over the official NVIDIA Samples [5] implementation. In the single-option pricing regime, our method delivers a $23\times$ speedup over the NVIDIA Sam-

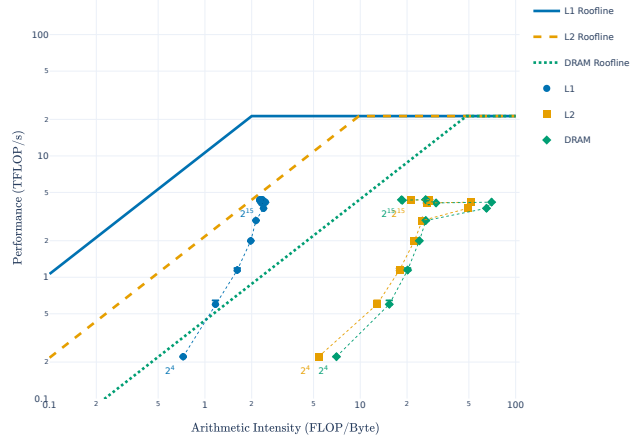


Fig. 6. Roofline plot of our method for the bulk option pricing task ($B = 1,000$, $N = 2^4, 2^5, \dots, 2^{15}$). The plot shows achieved performance on the first kernel launch for each calculation against theoretical peak performance for L1 cache (blue), L2 cache (orange), and DRAM (green). Each reported value is computed from the median execution time of 50 independent runs following warm-up iterations. A 99% confidence interval is displayed for each data point for both the arithmetic intensity and performance, although always within 5% of the median.

Ablated Optimization	Time (ms)
None (Baseline)	80.057 [79.967; 80.150]
Shuffle Intrinsic	98.458 [98.408; 98.511]
Pruning of OTM Nodes	104.75 [104.71; 104.79]
Adaptive Autotuning	110.80 [110.76; 110.87]
Temporal Unrolling	278.81 [277.96; 279.68]
Emulated-Double Arithmetic	305.18 [305.13; 305.23]
Exercise Value Caching	9,665.3 [9,661.6; 9,677.4]

Table 2. Ablation Study. Impact of individual optimizations on execution time ($N = 20,000$, $B = 100$). The baseline (first row) represents the fully optimized implementation. Subsequent rows quantify the performance degradation when specific features are disabled. We report the median of 50 independent runs following warm-up iterations. A 99% confidence interval is reported for each value.

ples [5] implementation at $N = 1,024$ and scales to binomial trees up to $N = 16,384$ time steps, where we record a $133\times$ speedup over a naive GPU implementation and a $736\times$ speedup over the industry standard Quantlib CPU baseline. Overall, our approach attains up to 77% and 76% of the theoretical peak compute and memory throughput, respectively, on a consumer-grade GPU.

6. REFERENCES

- [1] Fischer Black and Myron Scholes, “The Pricing of Options and Corporate Liabilities,” *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973.
- [2] John C. Cox, Stephen A. Ross, and Mark Rubinstein, “Option Pricing: A Simplified Approach,” *Journal of Financial Economics*, vol. 7, no. 3, pp. 229–263, 1979.
- [3] The QuantLib contributors, “QuantLib: A Free/Open-Source Library for Quantitative Finance,” <https://www.quantlib.org/>, 2025, Accessed: 2025-12-27.
- [4] Craig Kolb and Matt Pharr, “Option Pricing on the GPU,” in *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation*, Matt Pharr and Randima Fernando, Eds., chapter 45, pp. 719–731. Addison-Wesley Professional, Boston, MA, USA, 2005.
- [5] Victor Podlozhnyuk, “Binomial Option Pricing,” Whitepaper, NVIDIA Corporation, 2008, Included in NVIDIA CUDA Samples. https://github.com/NVIDIA/cuda-samples/blob/master/Samples/5_Domain_Specific/binomialOptions/doc/binomialOptions.pdf, Accessed: 2026-01-04.
- [6] Narayan Ganesan, Roger D. Chamberlain, and Jeremy Buhler, “Acceleration of Binomial Options Pricing via Parallelizing along Time-Axis on a GPU,” in *Proceedings of the Symposium on Application Accelerators in High Performance Computing*, Champaign, IL, USA, 2009.
- [7] John C. Hull, *Options, Futures, and Other Derivatives*, Pearson Education Limited, Harlow, UK, 10th edition, 2018.
- [8] NVIDIA Corporation, “NVIDIA RTX Blackwell GPU Architecture Whitepaper,” <https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>, May 2025, Accessed: 2026-01-04.
- [9] Andrew Thall, “Extended-Precision Floating-Point Numbers for GPU Computation,” in *ACM SIGGRAPH 2006 Research Posters*, New York, NY, USA, 2006, SIGGRAPH '06, p. 52, Association for Computing Machinery.
- [10] Nicholas J. Higham, *Accuracy and Stability of Numerical Algorithms*, SIAM, Philadelphia, PA, 2nd edition, 2002.
- [11] Jean-Yves Le Boudec, *Performance Evaluation of Computer and Communication Systems*, EPFL Press, Lausanne, Switzerland, 2010.
- [12] NVIDIA Corporation, “Profiling Guide – Nsight Compute 13.0 Documentation, Section 2.3.2: Metrics Structure,” <https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html#metrics-structure>, 2024, Accessed: 2026-01-09.